

Lab 4: Defender - Edited Chat Record

Raksha V | PES2UG24CS396 | 30 September 2026

Scope: This is an edited reconstruction of the lab discussion, not a verbatim conversation export. User messages and assistant replies below are condensed; uploaded screenshots and videos are described. The final submitted code is reproduced in the appendix. Keep the complete original chat export alongside this record for submission.

Assignment and setup

User: Shared Lab_4_VibeCoding_Student_handout.pdf and https://github.com/SETAPESU26/28_defender.

Assistant: Reviewed the handout: work individually; clone or fork the assigned repository; run the original Python game; record 10 seconds before editing; fix broken code and implement README features one by one; record 10 seconds after changes; push to a personal repository. Upload videos, updated code, and chat history as DOC/PDF under Lab-4. Do not raise a PR to SETAPESU26. Repository retrieval failed from the assistant environment, so the user was asked to share README and source.

User: Cloned successfully into C:\Users\Raksha\28_defender. The folder contained game.py and README.md.

Assistant: Requested the output of type README.md and type game.py.

User: Uploaded Pasted text(1).txt containing the README and original code.

Assistant: Identified four tasks: absolute world-coordinate radar, sky colour per wave, rescue celebration, and score-based bonus lives. Instructed the user to install pygame and run the original game before making changes.

User: Uploaded Pasted text(2).txt with installation output. Pygame compilation failed under Python 3.14 with missing distutils.msvccompiler / setuptools._distutils.msvccompiler.

Assistant: Suggested pygame-ce, which imports as pygame, using a prebuilt package to avoid the failed source build.

```
py -m pip install --only-binary=:all: pygame-ce
py game.py
```

User: okay video is done next

Radar repair

Assistant: Close the game and open notepad game.py. Replace the radar horizontal mapping line, preserving indentation.

```
# Before
rx = self.screen_x(x) % VIEW_W
# After
rx = (x % WORLD_W) / WORLD_W * VIEW_W
```

Assistant: The old line used coordinates relative to the player. The replacement scales absolute positions from the 3200-unit world onto the 800-pixel radar. Test by moving left/right: the white player blip moves while grounded green humanoid blips stay at their world positions.

User: should i make this video

Assistant: No separate radar video is required. Keep the before video and record the after video once all changes are complete.

User: Uploaded a gameplay screenshot; the image initially did not load.

Sky colour feature

Assistant: Replace sky_color with a five-colour palette indexed by wave. Wave 1 remains dark blue, Wave 2 is purple, Wave 3 teal, Wave 4 dark red, and Wave 5 lighter blue; later waves repeat the palette.

```
def sky_color(wave):
    """Change the sky colour as waves progress."""
    colors = [
        (5, 5, 20),
        (25, 10, 45),
        (10, 35, 50),
        (45, 15, 25),
        (35, 35, 60),
    ]
    return colors[(wave - 1) % len(colors)]
```

User: CHECK THIS - uploaded another gameplay screenshot.

Assistant: Successfully opened the screenshot. It showed Score 0, Lives 0, Wave 1 and GAME OVER. This is normal at zero lives; press R to restart. The radar blips were spread across the strip, consistent with the fix, but movement needed testing. Wave 1 should still have a dark sky. Defeat all enemies to reach Wave 2 and check the colour change.

User: ok next

Rescue celebration

Assistant: Implement a two-second +500 RESCUED! popup. The existing update code already awards 500 points, so do not add the score again in the hook.

```
def on_humanoid_rescued(humanoid):
    """Display a rescue celebration for two seconds."""
    humanoid.rescue_timer = 2.0
```

Assistant: Add timer decay at the beginning of Humanoid.update, before the falling-state check.

```
self.rescue_timer = max(
    0.0, getattr(self, "rescue_timer", 0.0) - dt
)
```

Assistant: In Game.draw, inside the visible-humanoid block and after drawing the humanoid, render the popup while its timer is positive.

```
if getattr(humanoid, "rescue_timer", 0.0) > 0:
    label = self.font.render(
        "+500 RESCUED!", True, (255, 255, 120)
    )
    screen.blit(
        label,
        label.get_rect(center=(sx, humanoid.y - 30))
    )
```

Assistant: To test, shoot a lander carrying a humanoid, then catch the falling humanoid with the ship before it lands. The game places it safely on the ground and shows the popup there.

User: ok next

Bonus life feature

Assistant: Return 10000 from `bonus_life_threshold`. The existing game logic awards one extra life when the score crosses the next milestone and records rewarded milestones.

```
def bonus_life_threshold():  
    """Award an extra life at each 10,000-point milestone."""  
    return 10000
```

Assistant: Save and run `py -m py_compile game.py`, then `py game.py`. No output from the compile command means the syntax check passed. Share the updated code for review before recording.

User: Uploaded Pasted text(3).txt with the updated game code.

Assistant: Reviewed all four edits and their indentation. The source passed Python compilation. Pygame was unavailable in the assistant runtime, so gameplay was not executed there. The user still needed to verify the rescue popup and bonus life during gameplay.

Video review

User: Uploaded Defender 2026-09-30 09-02-32.mp4.

Assistant: Checked duration and sampled frames. Duration was 47.177867 seconds. Frames showed world radar blips, shooting and score increases, progression to Wave 2 with a purple sky, and game over. No rescue popup was seen in sampled frames. The score reached 1650, so a 10000-point bonus life was not demonstrated. Suggested a 10-second clip around 14-24 seconds to capture the wave transition; this does not demonstrate every feature.

Repository and submission

User: Which one? Shared the GitHub dashboard.

Assistant: Initially suggested PES2UG24CS396-pes-vcs based on its name, asking the user to check for previous lab folders.

User: Shared that repository README.

Assistant: Corrected the suggestion: it was a separate C version-control-system project, not the intended lab repository. Asked for the repository containing earlier labs.

User: NO LAB REPO

Assistant: Guided creation of PES2UG24CS396-SE-Lab with a README, then creation of Lab-4/README.md containing name, SRN, assigned repository, and the four changes.

User: Shared a new-file screenshot.

Assistant: Noticed Lab-4/README.md had been typed into the left search box, while the actual filename field was blank. Directed the user to enter it in Name your file and commit directly to main.

User: DONE

Assistant: Directed upload of updated game.py and before/after 10-second MP4 videos inside Lab-4, followed by a commit.

User: Shared the Lab-4 folder screenshot.

Assistant: Confirmed the folder existed and pointed to Add file > Upload files.

User: NEXT - shared the uploaded files screenshot.

Assistant: Confirmed README.md, game.py, before.mp4.mp4 and after.mp4.mp4 were present. Noted duplicate extensions do not alter content. Asked the user to upload the complete chat history as PDF, including prompts, replies and code.

User: can u give that pdf

Submission location: <https://github.com/Raksha269/PES2UG24CS396-SE-Lab/tree/main/Lab-4>

Verification limits: Code review and compilation were performed. Gameplay screenshots and sampled video frames provide evidence for radar display and Wave 2 colour. Rescue popup timing and bonus-life behaviour were not verified in a running game by the assistant. Uploaded GitHub video lengths were not independently checked.

Appendix: Updated game.py

Source supplied by the user in Pasted text(3).txt, reproduced without functional changes. Long lines are wrapped for readability.

```
import math
import random
import pygame

VIEW_W, VIEW_H = 800, 560
WORLD_W = 3200
RADAR_H = 56
PLAY_TOP = RADAR_H + 6
HUMANOID_COUNT = 6
FALL_LIMIT = 160
PHASES = [random.uniform(0, math.tau) for _ in range(3)]

def sky_color(wave):
    """Change the sky colour as waves progress."""
    colors = [
        (5, 5, 20),      # Wave 1: dark blue
        (25, 10, 45),    # Wave 2: purple
        (10, 35, 50),    # Wave 3: teal
        (45, 15, 25),    # Wave 4: dark red
        (35, 35, 60),    # Wave 5: lighter blue
    ]
    return colors[(wave - 1) % len(colors)]

def on_humanoid_rescued(humanoid):
    """Display a rescue celebration for two seconds."""
    humanoid.rescue_timer = 2.0

def bonus_life_threshold():
    """Award an extra life at each 10,000-point milestone."""
    return 10000

def wrap_delta(a, b):
    """Shortest signed distance from world x=a to world x=b on a wrapping world."""
    return (b - a + WORLD_W / 2) % WORLD_W - WORLD_W / 2

def ground_y(x):
    angle = math.tau * x / WORLD_W
    return 505 - 25 * math.sin(3 * angle + PHASES[0]) - 14 * math.sin(7 * angle + PHASES[1]) - 6 *
    math.sin(13 * angle + PHASES[2])

class Humanoid:
    def __init__(self, x):
        self.x, self.y = x, ground_y(x) - 8
        self.state = "ground"
        self.fall_from = self.y
        self.vy = 0.0

    def update(self, dt):
        self.rescue_timer = max(
            0.0, getattr(self, "rescue_timer", 0.0) - dt
        )
        if self.state == "falling":
            self.vy += 300 * dt
            self.y += self.vy * dt
            floor = ground_y(self.x) - 8
            if self.y >= floor:
                self.y = floor
                self.vy = 0
                self.state = "dead" if floor - self.fall_from > FALL_LIMIT else "ground"

class Lander:
    def __init__(self, x):
        self.x, self.y = x, PLAY_TOP + 20
        self.target = None
        self.mutant = False

    def pick_target(self, humanoids, landers):
        free = [h for h in humanoids if h.state == "ground" and not any(h is l.target for l in
        landers)]
        self.target = random.choice(free) if free else None
```

```

def update(self, dt, player, humanoids, landers, wave):
    if self.mutant:
        dx = wrap_delta(self.x, player.x)
        dy = player.y - self.y
        dist = math.hypot(dx, dy) or 1
        speed = 110 + wave * 10
        self.x = (self.x + dx / dist * speed * dt + random.uniform(-40, 40) * dt) % WORLD_W
        self.y += dy / dist * speed * dt
        return
    if self.target is None or self.target.state not in ("ground", "carried"):
        self.target = None
        self.pick_target(humanoids, landers)
    target = self.target
    if target is None:
        dx, dy = wrap_delta(self.x, player.x), player.y - self.y
        dist = math.hypot(dx, dy) or 1
        self.x = (self.x + dx / dist * 60 * dt) % WORLD_W
        self.y += dy / dist * 60 * dt
    elif target.state == "carried":
        self.y -= 45 * dt
        target.x, target.y = self.x, self.y + 20
        if self.y <= PLAY_TOP + 8:
            humanoids.remove(target)
            self.target, self.mutant = None, True
    else:
        dx, dy = wrap_delta(self.x, target.x), target.y - 18 - self.y
        dist = math.hypot(dx, dy)
        if dist < 6:
            target.state = "carried"
        else:
            self.x = (self.x + dx / dist * 70 * dt) % WORLD_W
            self.y += dy / dist * 70 * dt

class Player:
    def __init__(self):
        self.x, self.y, self.vx, self.facing = WORLD_W / 2, 250.0, 0.0, 1
        self.invulnerable, self.cooldown = 1.5, 0.0

    def update(self, dt, keys):
        thrust = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        if thrust:
            self.facing = thrust
        self.vx = max(-520, min(520, self.vx + thrust * 900 * dt))
        self.vx *= 1 - min(1.0, 0.8 * dt)
        self.x = (self.x + self.vx * dt) % WORLD_W
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * 260 * dt
        self.y = max(PLAY_TOP + 10, min(VIEW_H - 30, self.y))
        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.cooldown -= dt

class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.player = Player()
        self.humanoids = [Humanoid(i * WORLD_W / HUMANOID_COUNT + 100) for i in
range(HUMANOID_COUNT)]
        self.landings, self.bullets = [], []
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.bonus_awaited = 0
        self.start_wave()

    def start_wave(self):
        self.to_spawn = 3 + self.wave * 2
        self.spawn_timer = 1.0
        while len(self.humanoids) < HUMANOID_COUNT:
            self.humanoids.append(Humanoid(random.uniform(0, WORLD_W)))

    def shoot_lander(self, lander):
        self.landings.remove(lander)
        if lander.target is not None and lander.target.state == "carried":
            lander.target.state = "falling"
            lander.target.fall_from = lander.target.y
        self.score += 150

    def update(self, dt, keys):
        if self.state != "play":
            return
        player = self.player

```

```

player.update(dt, keys)
threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
if keys[pygame.K_SPACE] and player.cooldown <= 0:
    player.cooldown = 0.18
    self.bullets.append({"x": player.x + player.facing * 20, "y": player.y, "dir":
player.facing, "life": 0.7})
    self.spawn_timer -= dt
    if self.to_spawn > 0 and self.spawn_timer <= 0:
        self.landings.append(Lander(random.uniform(0, WORLD_W)))
        self.to_spawn -= 1
        self.spawn_timer = 1.5
    for lander in self.landings:
        lander.update(dt, player, self.humanoids, self.landings, self.wave)
    for humanoid in self.humanoids[:]:
        humanoid.update(dt)
        if humanoid.state == "dead":
            self.humanoids.remove(humanoid)
        elif humanoid.state == "falling" and abs(wrap_delta(player.x, humanoid.x)) < 24 and
abs(humanoid.y - player.y) < 24:
            humanoid.state, humanoid.y = "ground", ground_y(humanoid.x) - 8
            self.score += 500
            on_humanoid_rescued(humanoid)
    self.update_bullets(dt)
    if player.invulnerable <= 0:
        for lander in self.landings:
            if abs(wrap_delta(player.x, lander.x)) < 22 and abs(lander.y - player.y) < 18:
                self.lives -= 1
                self.player = Player()
                if self.lives <= 0:
                    self.state = "lose"
                break
    if self.to_spawn == 0 and not self.landings:
        self.wave += 1
        self.start_wave()

def update_bullets(self, dt):
    for bullet in self.bullets:
        bullet["x"] = (bullet["x"] + bullet["dir"] * 900 * dt) % WORLD_W
        bullet["life"] -= dt
        for lander in self.landings[:]:
            if abs(wrap_delta(bullet["x"], lander.x)) < 16 and abs(bullet["y"] - lander.y) < 12:
                self.shoot_lander(lander)
                bullet["life"] = 0
                break
    self.bullets = [b for b in self.bullets if b["life"] > 0]

def screen_x(self, x):
    return VIEW_W / 2 + wrap_delta(self.player.x, x)

def draw_radar(self, screen):
    pygame.draw.rect(screen, (10, 10, 30), (0, 0, VIEW_W, RADAR_H))
    pygame.draw.rect(screen, (90, 90, 140), (0, 0, VIEW_W, RADAR_H), 1)
    blips = [(h.x, h.y, (90, 230, 120)) for h in self.humanoids]
    blips += [(l.x, l.y, (255, 90, 90) if l.mutant else (230, 200, 60)) for l in self.landings]
    blips.append((self.player.x, self.player.y, (255, 255, 255)))
    for x, y, color in blips:
        rx = (x % WORLD_W) / WORLD_W * VIEW_W
        ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H - 8) + 4
        pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))

def draw(self, screen):
    screen.fill(sky_color(self.wave) or (5, 5, 20))
    points = [(sx, ground_y(self.player.x + sx - VIEW_W / 2)) for sx in range(0, VIEW_W + 8, 8)]
    pygame.draw.polygon(screen, (110, 70, 40), points + [(VIEW_W, VIEW_H), (0, VIEW_H)])
    pygame.draw.lines(screen, (230, 150, 60), False, points, 2)
    for humanoid in self.humanoids:
        sx = self.screen_x(humanoid.x)
        if -20 < sx < VIEW_W + 20:
            pygame.draw.rect(screen, (90, 230, 120), (sx - 3, humanoid.y - 10, 6, 14))
            if getattr(humanoid, "rescue_timer", 0.0) > 0:
                label = self.font.render(
                    "+500 RESCUED!", True, (255, 255, 120)
                )
                screen.blit(
                    label,
                    label.get_rect(center=(sx, humanoid.y - 30))
                )
    for lander in self.landings:
        sx = self.screen_x(lander.x)
        if -20 < sx < VIEW_W + 20:

```

```

        color = (255, 90, 90) if lander.mutant else (230, 200, 60)
        pygame.draw.ellipse(screen, color, (sx - 14, lander.y - 9, 28, 18))
    for bullet in self.bullets:
        sx = self.screen_x(bullet["x"])
        pygame.draw.line(screen, (255, 255, 200), (sx - 8, bullet["y"]), (sx + 8, bullet["y"]),
2)
    player = self.player
    if player.invulnerable <= 0 or int(player.invulnerable * 10) % 2 == 0:
        f, cx = player.facing, VIEW_W / 2
        pygame.draw.polygon(screen, (240, 240, 250), [(cx + f * 18, player.y), (cx - f * 14,
player.y - 8), (cx - f * 14, player.y + 8)])
    self.draw_radar(screen)
    hud = self.font.render(f"Score {self.score} Lives {self.lives} Wave {self.wave} Humanoids
{len(self.humanoids)}", True, (240, 240, 240))
    screen.blit(hud, (10, RADAR_H + 4))
    if self.state == "lose":
        label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
        screen.blit(label, label.get_rect(center=(VIEW_W // 2, VIEW_H // 2)))

def main():
    pygame.init()
    screen = pygame.display.set_mode((VIEW_W, VIEW_H))
    pygame.display.set_caption("Defender")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()

if __name__ == "__main__":
    main()

```

Conversation update: fork and submission

Raksha V | PES2UG24CS396 | Updated 30 September 2026

Scope: This addendum is an edited reconstruction of the recent conversation. Together with the earlier pages, it is an edited chat record, not a complete verbatim chat export. Assistant replies below are summarized.

User: For submission we have to upload github repo link and folder containing deliverables

Assistant: The submission needs a repository link and a folder containing the requested deliverables: updated game.py, before and after videos, and chat history as DOC or PDF. Use a ZIP if the submission portal requires one.

User: no for link we have to commit changes from that link only

Assistant: Uploading changes through GitHub creates a commit. The initial response assumed the course deliverables repository was the required code link; the next clarification corrected that assumption.

User: no the original link we have to fork smthng and all

Assistant: Fork the assigned SETAPESU26/28_defender repository into your own GitHub account, upload the updated game.py to your fork, and commit it to main. Do not open a pull request to the original repository. Keep the requested deliverables together in the Lab-4 folder.

User: fork done

Assistant: Open your fork, upload the updated game.py from C:\Users\Raksha\28_defender, and commit the change to main. Check that the file contains the updated radar mapping and other game changes.

User: shared a screenshot of the committed game.py

Assistant: The screenshot showed game.py in Raksha269/28_defender with the updated sky-color function and an upload commit. The fork link can be used for the code submission. The conversation PDF still needs to be included with the deliverables.

User: should we include the conversation report

Assistant: Yes. The assignment requests chat history as DOC or PDF. Include it in Lab-4 and in the submitted deliverables folder or ZIP. The edited report is supplementary and does not replace a complete history export if the instructor requires the full conversation.

User: can u update the chat pdf and send

Assistant: The PDF is updated with the fork, commit, and submission discussion. It remains an edited chat record rather than a verbatim export.

Repository references

Assigned repository: https://github.com/SETAPESU26/28_defender

Personal fork for updated code: https://github.com/Raksha269/28_defender

Course deliverables folder: <https://github.com/Raksha269/PES2UG24CS396-SE-Lab/tree/main/Lab-4>

Submission status and checks

The user confirmed the fork was created and supplied evidence of the game.py commit. Earlier screenshots showed game.py and before/after video files in Lab-4. This record does not verify that the revised PDF has been uploaded or that the final submission has been completed. Check that the submitted videos meet the requested 10-second length and include the required chat history.